

ina M.

re pour connecter les grimpeurs, les clubs, les lieux et les

Événements clubs

RNCP39583 · BLOC 2

1
OBJETS DÉCLARÉS

Concevoir et développer des applications logicielles

Spity · Prototype web pour la communauté escalade

Lieu

Profil actif

lina.demo@spity.fr

Disciplines

bloc vole

Matériel

1 x Grand Rel

Candidat

Dorian Joly

Version

Spity 0.1.0

Livrable

Dossier BC02 avec annexes

Date

23 juillet 2026

Il y a 18 min

Événement

Prochains ren

Sortie fal

Club Alpin

e cherche quelqu'un pour travailler les profils déversants et filmer

Sommaire du dossier

Cette version destinée au jury met en avant mes quatre compétences déterminantes, puis conserve mon retour d'expérience, les 26 critères officiels et neuf annexes visuelles. Les documents techniques complémentaires restent accessibles par les liens du dossier et dans [docs/bc02/](#) .

01 Mon dossier de validation BC02 - Spity

02 Mes quatre compétences déterminantes du bloc 2

03 Mon retour d'expérience et mes choix techniques - BC02

04 Index des critères et des preuves BC02

05 Annexes visuelles et preuves d'exécution - BC02

Mon dossier de validation BC02 - Spity

Identification

Champ	Valeur
Candidat	Dorian Joly
Certification	Expert en développement logiciel - RNCP39583
Bloc	BC02 - Concevoir et développer des applications logicielles
Projet	Spity, réseau social pour la communauté escalade
Prototype évalué	Application web responsive, version <code>0.1.0</code>
Branche de travail	<code>develop</code>
Release stable	<code>v0.1.0</code>
Date du dossier	23 juillet 2026
Dépôt	github.com/Dorianyloj/spity

Accès direct pour le jury

Accès	Valeur
Application	https://spity.fr
Santé et version déployée	https://spity.fr/api/health
Code source et historique Git	github.com/Dorianyloj/spity
Révision déployée	<code>49c4ea0ffa34b35e9ad5bc2e1a838eb82eb0b8ef</code>
Compte grimpeur	<code>lina.demo@spity.local</code>
Compte club	<code>club.demo@spity.local</code>
Mot de passe commun	<code>SpityDemo2026!</code>

Cette instance est réservée à la démonstration : elle ne contient que des données fictives et peut être réinitialisée. Les deux comptes permettent au jury de manipuler les parcours principaux sans créer de compte ni accéder à l'administration du serveur.

Déclaration de périmètre

Dans ce dossier, je présente le travail que j'ai réalisé sur Spity pour les neuf compétences du bloc BC02. Je me suis appuyé sur une version fonctionnelle, versionnée et testée du projet. Mon [audit de conformité aux documents officiels](#) recense neuf compétences sur neuf, seize livrables sur seize et vingt-six critères sur vingt-six couverts par une preuve. J'ai volontairement séparé :

- la vision produit complète de Spity ;

- le prototype BC02 effectivement développé ;
- les fonctions testées et démontrables ;
- les limites et évolutions encore nécessaires.

Je ne présente pas comme terminée une fonction absente du périmètre ou encore non persistante. Pour chaque compétence, j'ai essayé de fournir une explication, un fichier ou un artefact, un résultat vérifiable et, lorsque c'est nécessaire, une limite clairement annoncée.

Résumé du projet

J'ai imaginé Spity à partir d'un constat simple : un grimpeur utilise souvent plusieurs services pour rechercher un partenaire, trouver un lieu ou participer à une sortie. J'ai choisi de réunir ces usages dans un même prototype avec deux rôles :

- le **grimpeur**, qui gère son profil et son matériel, recherche des partenaires, traite des demandes et s'inscrit à des événements ;
- le **club**, qui gère sa fiche, publie des événements et suit les participants.

Un visiteur peut créer un compte ou se connecter. J'ai ensuite fait dépendre les autorisations, la capacité des événements et les données visibles du rôle et de la session active.

Périmètre démontrable

ID	Fonction
F01	Inscription, connexion, session et déconnexion.
F02	Création, consultation et modification des profils grimpeur et club.
F03	Recherche de partenaires par texte, discipline, niveau, disponibilité et environnement.
F04	Envoi, acceptation, refus et historique des demandes de partenariat.
F05	Création, modification et annulation d'un événement par son club.
F06	Consultation des événements et des capacités restantes.
F07	Inscription et désinscription d'un grimpeur.
F08	Consultation des participants par le club organisateur.
F09	Validation, contrôle de session, rôle, origine, quota et réponses sûres.
F10	Structure sémantique, clavier, reflow mobile et mouvement réduit.

J'ai laissé hors du prototype évalué le fil social persistant, les likes/commentaires, la carte interactive, les topos collaboratifs, les notifications, la récupération de mot de passe et les parcours RGPD autonomes. Ces fonctions appartiennent à la vision complète, mais les intégrer sans pouvoir les tester correctement aurait fragilisé la démonstration.

Architecture synthétique

```

Navigateur React / composants accessibles
    |
    v
Next.js App Router et Route Handlers
    |
    Zod + règles métier TypeScript
    |
    Dépôts Drizzle ORM
    |
    v
    MariaDB 11.4
  
```

J'ai utilisé TypeScript strict de bout en bout. J'ai retenu Next.js pour réunir les pages, les composants serveur et les API dans un même projet, puis produire un artefact standalone sous Node.js 22. J'utilise Zod pour contrôler les données à l'exécution, Drizzle pour structurer les accès et les migrations, et MariaDB pour conserver les relations métier. Docker sépare l'application, les migrations et la base. Enfin, GitHub Actions reconstruit et teste chaque révision avant le staging.

Résultats vérifiés

Axe	Résultat de référence
Analyse statique	ESLint et TypeScript réussis.
Tests unitaires	126 tests réussis dans 23 suites.
Couverture globale mesurée	62,56 % lignes/instructions, 77,67 % branches et 55,06 % fonctions sur tout <code>src/</code> .
Intégration	11 résultats HTTP/MariaDB réussis sur une base migrée à neuf.
Recette	6 scénarios Playwright couvrant F01 à F10, sans retry ni skip.
Sécurité	0 vulnérabilité haute ou critique de production ; dix catégories OWASP analysées.
Accessibilité authentifiée	Dix états à 100 % dans l'audit automatisé.
Lighthouse public	Performance 97 à 99, accessibilité 100, SEO 100.
Responsive	Reflow vérifié à 360 px sans défilement horizontal incohérent.
Livraison	Release <code>v0.1.0</code> , images immuables, bundle, manifeste et SHA-256.
Dernière CI probante	Run 29988586736 , cinq jobs réussis dont staging, couverture globale et audit de production.
Annexes visuelles	Neuf captures : produit desktop/mobile, rapports Jest/Playwright, historique Git, pipeline CI et release.

Couverture des compétences

C2.1.1 - Environnements et déploiement

J'ai séparé les environnements de développement, de test, de staging et de production. J'ai défini les outils, les variables, les seuils et l'ordre des opérations. Le staging construit des images identifiées par SHA seulement après les contrôles qualité, puis vérifie les migrations et la santé de l'application dans un environnement isolé.

Preuve principale : [environnements, qualité et déploiement](#).

C2.1.2 - Intégration continue

J'ai construit le pipeline autour d'une installation verrouillée, du lint, du typage, de Jest, de l'audit, des configurations Compose, du build, de MariaDB, de l'accessibilité, de Lighthouse et de la recette standalone. Je voulais qu'un échec bloque réellement la promotion. Le run `29749715001`, où le staging a été ignoré après un échec, montre que ce mécanisme fonctionne.

Preuve principale : [protocole d'intégration continue](#).

C2.2.1 - Prototype maintenable

J'ai organisé le code par fonctionnalité afin de séparer la présentation, les API, les règles métier et les données. Les composants partagés et les contrats Zod m'ont permis de limiter la duplication. J'ai vérifié les user stories retenues sur desktop et sur mobile.

Preuve principale : [architecture et prototype](#).

C2.2.2 - Harnais unitaire

J'utilise Jest pour les validateurs, le parseur de matériel, les règles d'événements et de matching, le quota, le contrôle d'origine, les en-têtes, les principaux composants interactifs et le contrat Drizzle. Après une première mesure trop ciblée, j'ai étendu la configuration à tout `src/`. J'atteins 62,56 % des lignes et instructions. Les seuils globaux bloquent désormais la CI ; les tests MariaDB et Playwright complètent cette couverture.

Preuves principales : [harnais Jest](#) et [tests d'intégration](#).

C2.2.3 - Sécurité et accessibilité

J'ai relié mes mesures de sécurité aux dix catégories OWASP 2025. Pour l'accessibilité, j'ai retenu le RGAA 4.1.2, adapté au contexte français. Je ne me suis pas limité aux scores automatisés : j'ai aussi vérifié le clavier, le focus, la réduction des mouvements et le reflow mobile.

Preuves principales : [sécurité OWASP](#) et [accessibilité RGAA](#).

C2.2.4 - Versions et déploiements

J'utilise Git, Semantic Versioning, un changelog et des images identifiées par SHA pour relier une évolution à l'artefact livré. La release stable reprend le même candidat que celui testé et fournit un bundle de déploiement autonome.

Depuis le 23 juillet 2026, le jury peut manipuler la version déployée sur `spity.fr` sans installer le projet ni accéder au serveur. Le manuel, les comptes grimpeur et club prêts à l'emploi, la route de santé et les contrôles navigateur desktop/mobile rendent le parcours autonome et vérifiable.

Preuve principale : [versions et déploiements](#).

C2.3.1 - Cahier de recettes

J'ai construit le cahier de recettes autour des fonctions F01 à F10, des deux rôles, des cas alternatifs, de la sécurité, de la structure et du mobile. Les scénarios sont automatisés et exécutés sur une MariaDB créée puis migrée à neuf.

Preuve principale : [cahier de recettes](#).

C2.3.2 - Correction des bogues

J'ai conservé sept anomalies réellement rencontrées, puis je les ai qualifiées, corrigées et retestées. Dans le registre, je distingue un défaut du produit d'une erreur du harnais de test. Je n'ai baissé aucun seuil et je n'ai pas affaibli les cookies de production pour rendre la CI verte.

Preuve principale : [plan de correction](#).

C2.4.1 - Documentation d'exploitation

J'ai rédigé trois manuels séparés pour le déploiement, l'utilisation et la maintenance de Spity. Ils décrivent les langages, les technologies, les commandes, les rôles, les diagnostics, les sauvegardes, les mises à jour et les retours arrière.

Preuves principales : [manuel de déploiement](#), [manuel d'utilisation](#) et [manuel de maintenance](#).

Démonstration que je prévois

1. Je présente le périmètre F01-F10 et les limites.
2. Je me connecte avec le compte grimpeur Lina.
3. Je montre son profil, son matériel, les filtres de matching et les lieux.
4. Je me connecte avec le compte club et je crée un événement de capacité **1**.
5. Je reviens au compte Lina pour réaliser l'inscription.
6. Je reviens au club pour afficher la participante puis annuler l'événement.
7. Je présente le run CI vert, le run bloqué et le registre de correction.
8. J'ouvre l'index des critères pour répondre aux questions du jury.

Limites et perspectives

Je présente Spity comme un prototype de certification, pas comme une plateforme publique déjà prête à accueillir des données réelles. Les principales limites que j'ai identifiées sont :

- une session pilote externe reste à organiser pour mesurer plus finement l'ergonomie réelle ;
- couverture des dépôts Drizzle et Route Handlers encore inférieure à celle des composants et règles métier ;
- récupération de mot de passe, export et suppression autonome de compte non exposés ;
- fil social, carte interactive et topos encore démonstratifs ou hors périmètre ;
- alertes modérées de dépendances suivies, sans vulnérabilité haute ou critique ;
- supervision, sauvegardes planifiées et test de restauration à industrialiser avant production publique.

Mes prochaines priorités sont la validation utilisateur, les parcours RGPD, la récupération de compte, la persistance sociale, la cartographie et le renforcement des tests des couches serveur.

Conclusion

Avec Spity, je montre une chaîne complète de conception et de développement : j'ai cadré le besoin, structuré l'architecture, développé un ensemble cohérent de fonctions, automatisé les contrôles, corrigé les anomalies, produit une version stable et rédigé la documentation d'exploitation. Je ne veux pas que ce dossier repose uniquement sur mes explications : chacune des neuf compétences et chacun des vingt-six critères renvoient à du code versionné, une commande, un livrable ou un résultat distant. Une future session avec des utilisateurs externes renforcera l'évaluation ergonomique sans remplacer les preuves déjà fournies.

L'[index des critères](#) constitue le point d'entrée pour l'évaluation. Les documents techniques liés fournissent les explications et preuves nécessaires à chaque ligne de la grille.

Les [annexes visuelles](#) regroupent les captures produites, les rapports de tests et les preuves GitHub utilisées pendant la soutenance.

Mes quatre compétences déterminantes du bloc 2

Les compétences C2.2.1, C2.2.2, C2.2.3 et C2.3.1 sont déterminantes pour la validation de mon bloc 2. Je ne veux donc pas seulement renvoyer vers du code ou annoncer qu'un contrôle est « conforme ». Dans ce chapitre, je reprends les indicateurs de la grille et j'explique ce que j'ai réellement réalisé, le résultat que j'ai obtenu et la preuve que je peux présenter au jury.

C2.2.1 - Concevoir un prototype fonctionnel, maintenable et sécurisé

Ce que je dois démontrer

La grille attend une architecture structurée, un prototype présenté et l'utilisation justifiée de frameworks et de paradigmes de développement. Elle vérifie aussi cinq résultats : le respect des bonnes pratiques, la réponse au besoin, la cohérence des fonctionnalités et des user stories, le fonctionnement des composants d'interface et la prise en compte de la sécurité.

Ce que j'ai conçu

J'ai conçu Spity comme une application web responsive utilisable sur ordinateur et mobile. J'ai volontairement limité le prototype à dix fonctions démontrables, de F01 à F10, plutôt que de simuler toute la vision produit. Le parcours principal relie l'inscription, le profil grimpeur, la recherche d'un partenaire, les demandes de partenariat et l'inscription à un événement. Le parcours club permet de créer son profil, publier un événement, suivre les participants, modifier la capacité et annuler l'événement.

J'ai retenu une architecture en couches dans un seul projet TypeScript :

1. les pages Next.js et les composants React présentent les données et les actions ;
2. les Route Handlers contrôlent la session, le rôle, l'origine et les entrées Zod ;
3. les fonctions métier TypeScript portent les règles de matching, d'événement et de capacité ;
4. les repositories Drizzle isolent la persistance ;
5. MariaDB garantit les relations, les contraintes uniques et les transactions.

Next.js et React répondent au besoin web et responsive. Tailwind et mon design system évitent de dupliquer les composants. React Hook Form et Zod donnent une validation cohérente entre les formulaires et l'API. Drizzle me permet de conserver des requêtes typées et des migrations versionnées. Je sépare les fonctionnalités dans `src/features/` afin qu'un changement sur les événements n'oblige pas à modifier le matching ou les profils.

Correspondance avec les indicateurs officiels

Indicateur de la grille	Ma réalisation vérifiable	Résultat
Bonnes pratiques, frameworks et paradigmes	TypeScript strict, App Router, composants réutilisables, validation Zod, règles métier pures, repositories Drizzle et migrations additives.	Couvert
Prototype fonctionnel répondant aux besoins	Deux rôles et dix fonctions F01-F10 reliés dans des parcours de bout en bout.	Couvert
Ensemble cohérent de fonctions et de user stories	F01-F04 couvrent l'accès et le matching ; F05-F08 couvrent les événements ; F09-F10 portent les exigences transverses.	Couvert
Composants d'interface présents et fonctionnels	Formulaires, navigation, filtres, cartes, états vides, boutons et retours d'erreur testés sur desktop et à 360 px.	Couvert
Exigences de sécurité	Session signée, rôles, contrôle du propriétaire, CSRF, quota, Zod, requêtes paramétrées, transactions et en-têtes HTTP.	Couvert

Je peux démontrer ce résultat sans préparation spéciale sur spity.fr : je me connecte comme grimpeur, je filtre les partenaires et je m'inscris à un événement ; je passe ensuite sur le compte club pour retrouver le participant et administrer l'événement. Les captures A1 à A4 montrent les interfaces desktop et mobile. L'[architecture détaillée](#) relie chaque user story à sa route, sa règle et sa preuve.

C2.2.2 - Développer un harnais de tests couvrant la majorité du code

Ce que je dois démontrer

La grille demande un jeu de tests unitaires portant sur les fonctionnalités demandées et couvrant la majorité du code développé. Pour éviter une couverture artificielle, ma configuration mesure tout `src/`, y compris les fichiers sans test, et pas seulement les modules faciles à tester.

Mon harnais de tests

J'utilise Jest, jsdom et React Testing Library. Mes 23 suites regroupent 126 tests. Elles contrôlent notamment :

- les schémas Zod et les entrées invalides ;
- les règles de matching et d'événements, dont les limites de capacité ;
- le contrôle CSRF, le quota et les en-têtes de sécurité ;
- le parseur de matériel et ses cas limites ;
- les composants partagés, les formulaires et la navigation clavier ;
- les tableaux de bord, profils, lieux, partenariats et événements ;
- le contrat des tables Drizzle nécessaires au prototype.

Le fichier `jest.config.ts` impose quatre seuils globaux. Si un seuil baisse, `npm run test:coverage` échoue et le job `Quality gates` bloque le staging.

Indicateur mesuré sur tout <code>src/</code>	Résultat	Seuil bloquant	Décision
Lignes	62,56 %	60 %	Conforme
Instructions	62,56 %	60 %	Conforme
Branches	77,67 %	75 %	Conforme
Fonctions	55,06 %	55 %	Conforme
Exécution	126 tests dans 23 suites	100 % des tests réussis	Conforme

La couverture des lignes, instructions, branches et fonctions dépasse donc la moitié du code mesuré. La capture A8 montre le rapport global produit par Jest. Je complète les tests unitaires avec une intégration HTTP/MariaDB et une recette Playwright, mais je ne les additionne pas artificiellement au pourcentage unitaire.

Le harnais a déjà détecté une vraie régression : la virgule décimale de `9,1 mm` était interprétée comme un séparateur de matériel. J'ai ajouté le cas de test, corrigé le découpage puis exécuté toute la suite. Cette anomalie montre que les tests protègent un comportement métier concret et ne servent pas uniquement à produire un chiffre. Le détail des suites et de la progression se trouve dans le [document du harnais](#).

Je garde une limite explicite : plusieurs Route Handlers et repositories sont moins couverts que les règles et composants. Le seuil global officiel est atteint, mais ma prochaine étape technique consiste à isoler davantage ces dépendances pour augmenter la couverture serveur sans remplacer les tests d'intégration.

C2.2.3 - Garantir l'évolutivité, la sécurité et l'accessibilité

Ce que je dois démontrer

La grille demande une présentation des mesures de sécurité couvrant les dix risques OWASP, le choix justifié d'un référentiel d'accessibilité et la preuve que le prototype respecte les exigences applicables de ce référentiel. J'ai aussi relié ces contrôles aux spécifications techniques et fonctionnelles du projet.

Sécurité mise en œuvre

J'ai utilisé l'OWASP Top 10:2025, version la plus récente au moment du dossier. La matrice complète est disponible dans mon [audit de sécurité](#). Les mesures ne sont pas seulement documentées : elles se retrouvent dans le code, les tests unitaires, les tests HTTP/MariaDB et la CI.

Catégorie OWASP	Mesure principale que j'ai mise en œuvre
A01 Contrôle d'accès	Session serveur, rôles, propriétaire/destinataire vérifiés, réponses 401/403 et contrôle CSRF.
A02 Mauvaise configuration	Variables validées, CSP, HSTS, <code>nosniff</code> , anti-frame et suppression de l'en-tête de technologie.
A03 Chaîne logicielle	Lockfile, <code>npm ci</code> , audit bloquant sur haut/critique et Dependabot npm/Actions.
A04 Cryptographie	bcrypt facteur 12, secret HMAC borné, comparaison constante et cookie <code>HttpOnly / Secure</code> .
A05 Injection	Zod, Drizzle paramétré, rendu React échappé et absence de SQL ou HTML concaténé.
A06 Conception non sûre	Transitions métier bornées, paire unique, transaction et verrou lors de la dernière place.
A07 Authentification	Erreur neutre, verrouillage du compte, quota 10/15 min et session signée expirante.
A08 Intégrité	Migrations et dépendances versionnées, réponses API typées, artefacts reliés au SHA.
A09 Journalisation	Événements JSON sans email, mot de passe ni jeton ; conservation des résultats CI.
A10 Cas exceptionnels	JSON invalide géré, réponses sans stack, transactions, healthcheck et error boundary.

J'annonce aussi les risques résiduels : quota mémoire mono-instance, alerting non centralisé, session non révocable avant expiration et deux alertes PostCSS modérées. Je préfère présenter ces limites et leur plan de traitement plutôt que revendiquer une sécurité de production que le prototype n'a pas encore.

Accessibilité mise en œuvre

J'ai choisi le RGAA 4.1.2 parce qu'il s'agit du référentiel français directement applicable à une interface web et qu'il donne des critères vérifiables sur la structure, le clavier, les formulaires, les couleurs et la consultation. Mon échantillon contient les pages publiques, huit états authentifiés et deux vues mobiles à 360 px.

Mon premier audit a révélé des défauts concrets : absence de lien d'évitement, retours non annoncés, contrastes insuffisants, hiérarchie de titres incorrecte et débordements sur mobile. J'ai corrigé ces points avec un lien vers le contenu principal, `role="alert"` et `aria-live`, des couleurs à plus de 4,5:1, des titres ordonnés, une commande mobile de 44 x 44 px, un reflow sans largeur parasite et la prise en charge de `prefers-reduced-motion`.

Les résultats après correction sont les suivants : aucune violation axe sur l'échantillon, accessibilité Lighthouse à 100 % sur les trois pages publiques, huit états authentifiés et deux vues mobiles, et aucun débordement du document à 360 px. Ces contrôles sont bloquants dans la CI. Le [rapport RGAA](#) reprend les treize thèmes, les critères applicables, les corrections avant/après et les limites des outils automatiques.

L'évolutivité est assurée par la séparation des fonctionnalités, les composants partagés, les contrats Zod et les migrations additives. La conformité fonctionnelle est vérifiée par les mêmes fonctions F01-F10 que celles annoncées dans le périmètre : je ne teste pas un produit différent de celui que je présente.

C2.3.1 - Élaborer et exécuter le cahier de recettes

Ce que je dois démontrer

La grille vérifie que le cahier reprend toutes les fonctionnalités attendues et que les tests fonctionnels, structurels et de sécurité ont réellement été exécutés conformément au plan. Mon cahier ne se limite donc pas à une liste de scénarios futurs : il contient les préconditions, les actions, les résultats attendus, les résultats obtenus et la décision de recette.

Ma recette exécutée

J'ai relié les dix fonctions du périmètre à six scénarios Playwright exécutés sur le build standalone avec une MariaDB créée et migrée pour la recette.

Scénario	Fonctions et risques vérifiés	Résultat obtenu
REC-F01-001	Mot de passe faible, inscription, onboarding, session, déconnexion et reconnexion.	Conforme
REC-F02-001	Consultation et modification du profil grimpeur, puis création d'un profil club.	Conforme
REC-F03-F04-001	Filtres combinés, état vide, demandes, acceptation, refus et historique.	Conforme
REC-F05-F08-001	Création d'un événement, dernière place, confidentialité, capacité, désinscription et annulation.	Conforme
REC-F09-001	Accès anonyme, mauvais rôle, origine étrangère, UUID invalide, CSP et réponses sûres.	Conforme
REC-F10-001	Navigation clavier, titre, nom accessible, mouvement réduit et reflow à 360 px.	Conforme

La couverture fonctionnelle est complète pour le périmètre annoncé : F01 à F10 possèdent toutes une preuve navigateur et une preuve complémentaire unitaire ou d'intégration. Les contrôles structurels ajoutent ESLint, TypeScript strict et le build de 28 routes. Les contrôles de sécurité ajoutent l'audit des dépendances, les tests CSRF/quota et les autorisations HTTP. L'accessibilité ajoute axe et Lighthouse.

Le résultat de référence est de 6 scénarios réussis sur 6, sans test ignoré, instable ou rejoué. Les 126 tests unitaires, les 11 résultats d'intégration et les cinq jobs de la CI sont également réussis. La capture A9 montre le rapport Playwright et le run GitHub Actions 29831871915 confirme que le staging n'a commencé qu'après la réussite des quatre portes précédentes.

Les scénarios exécutables sont conservés dans [tests/acceptance/bc02-recipe.spec.ts](#). Le [cahier de recettes complet](#) documente les critères d'entrée, les données, la matrice F01-F10, les résultats et les artefacts. Ma décision de recette est donc « favorable pour le périmètre BC02 », avec les fonctions hors périmètre toujours annoncées comme telles.

Synthèse de validation

Compétence déterminante	Preuve centrale	Résultat
C2.2.1	Prototype desktop/mobile, architecture et parcours F01-F10	Couvert
C2.2.2	126 tests, couverture globale majoritaire et seuils CI	Couvert
C2.2.3	Matrices OWASP/RGAA, corrections et audits automatisés	Couvert sur le prototype
C2.3.1	Cahier F01-F10 et six scénarios Playwright exécutés	Couvert

Pendant la soutenance, je commencerai par ces quatre compétences. Pour chacune, je présenterai d'abord le besoin, puis une manipulation ou un résultat, et enfin le fichier de preuve. Cela me permet de montrer que le prototype, les tests, la sécurité, l'accessibilité et la recette correspondent au même périmètre fonctionnel.

Mon retour d'expérience et mes choix techniques - BC02

Ce chapitre complète la synthèse et l'index officiel. J'y explique comment j'ai réellement construit Spity, les décisions que j'ai prises et les problèmes que j'ai rencontrés. Mon objectif n'est pas de reprendre toute la documentation fichier par fichier, mais de montrer ma démarche de développeur et la manière dont j'ai relié le besoin, le code, les tests et la livraison.

1. Pourquoi j'ai choisi ce projet

J'ai choisi le sujet de l'escalade parce qu'il permet de travailler sur autre chose qu'un simple CRUD. Le besoin mélange des profils, de la recherche, des relations entre utilisateurs, des événements, des lieux et des informations qui peuvent avoir un impact sur la sécurité. Cela m'a obligé à réfléchir à la fois à l'expérience utilisateur, aux règles métier et à la qualité des données.

Au départ, la vision de Spity était très large : un réseau social, des topos collaboratifs, une carte, des salles, des falaises, des clubs, des événements et du matching entre grimpeurs. Je me suis rapidement rendu compte qu'il serait impossible de rendre toutes ces fonctions fiables dans le temps du bloc 2. J'ai donc choisi de démontrer un noyau cohérent plutôt qu'une accumulation d'écrans partiellement fonctionnels.

Le fil conducteur que j'ai retenu est le suivant : un grimpeur crée son profil, indique sa pratique et son matériel, recherche un partenaire, échange une demande, consulte un événement puis s'y inscrit. En parallèle, un club peut publier cet événement, régler sa capacité et voir les participants. Ces deux parcours utilisent la même session, la même base et les mêmes règles d'autorisation.

Ce choix me permet de présenter une chaîne de bout en bout :

1. une interface utilisable sur desktop et mobile ;
2. des Route Handlers qui valident les entrées ;
3. des règles métier testables séparément ;
4. une persistance MariaDB avec contraintes ;
5. des tests unitaires, d'intégration et de recette ;
6. un pipeline qui bloque réellement une version défectueuse.

Je considère que ce périmètre est représentatif du produit final. Les topos, la carte et le fil social restent dans la vision, mais je ne les présente pas comme terminés. Cette distinction est importante dans mon dossier : je préfère annoncer clairement ce qui manque plutôt que de donner une impression de complétude qui ne résisterait pas à la démonstration.

2. Comment j'ai cadré le MVP

J'ai transformé la vision initiale en dix fonctions démontrables, notées F01 à F10. Elles couvrent l'authentification, les profils, le matching, les partenariats, les événements, les inscriptions, les contrôles de sécurité et le responsive. Pour chaque fonction, j'ai associé une user story et un résultat observable dans le [périmètre fonctionnel](#).

Pour choisir ce qui entrait dans le MVP, j'ai utilisé trois questions simples :

- est-ce que la fonction sert directement un grimpeur ou un club ?
- est-ce que je peux la persister et la tester correctement ?
- est-ce qu'elle complète un parcours déjà commencé ?

Cette méthode m'a conduit aux décisions suivantes :

Sujet	Décision	Raison
Authentification et profils	Inclus	Point d'entrée obligatoire pour tous les parcours.
Matching et demandes	Inclus	Valeur principale pour le rôle grimpeur.
Événements et inscriptions	Inclus	Parcours complet reliant grimpeur, club et capacité.
Répertoire de lieux	Inclus en consultation	Utile à la démonstration, sans dépendre d'une API cartographique.
Carte interactive	Reportée	Nécessite le choix d'un fournisseur, des clés et une stratégie de quotas.
Fil social persistant	Reporté	Demande médias, modération, pagination et règles RGPD supplémentaires.
Topos collaboratifs	Reportés	Exigent une modération et une qualité de données plus poussées.
Notifications	Reportées	Dépendent d'un service asynchrone et d'un consentement utilisateur.

J'ai aussi séparé le périmètre du produit de celui de la certification. Le fichier `CADRAGE_PROJET.md` conserve l'ambition globale, tandis que le dossier BC02 décrit seulement ce que je peux exécuter et prouver. Cette séparation m'a aidé à éviter deux erreurs : modifier constamment le périmètre pendant le développement et écrire une documentation qui promet plus que le code.

Pour la soutenance, j'ai prévu une démonstration courte avec deux comptes. Je commence avec Lina, le compte grimpeur, puis je passe sur le compte du Club Alpin Lyon. Je peux ainsi montrer les autorisations différentes et faire évoluer la même donnée sous deux points de vue. La capacité d'un événement fixée à une place rend la règle métier visible immédiatement.

3. L'architecture que j'ai retenue

J'ai retenu Next.js avec l'App Router parce qu'il me permet de garder les pages, les composants serveur, les composants interactifs et les API dans un projet TypeScript unique. Cela réduit le nombre de dépôts à maintenir pour le prototype, tout en conservant une séparation par couche.

Mon organisation principale est orientée par fonctionnalité :

```
src/features/matching/
  components/
  lib/
  schemas.ts

src/features/events/
  components/
  lib/
  schemas.ts
```

Dans chaque feature, les composants gèrent l'affichage, les schémas Zod définissent les contrats, les règles pures portent les décisions métier et les repositories isolent Drizzle. Les pages de `src/app/` chargent l'utilisateur et les données initiales. Les Route Handlers de `src/app/api/` appliquent les contrôles HTTP et appellent la couche métier.

Je n'ai pas voulu mettre toute la logique dans les composants React. Par exemple, le filtrage des grimpeurs et les règles de capacité sont des fonctions indépendantes. Je peux donc les tester sans navigateur. De la même manière, les composants ne connaissent pas les colonnes MariaDB : ils manipulent des objets validés par les schémas.

J'ai utilisé TypeScript en mode strict. Lorsqu'une donnée vient d'une requête ou d'un JSON MariaDB, je la considère d'abord comme `unknown`. Elle doit passer par une validation ou un narrowing avant d'être utilisée. Cette contrainte ajoute du travail au début, mais elle m'a évité de propager des hypothèses implicites entre l'API et l'interface.

J'ai choisi les outils suivants pour des raisons précises :

Outil	Utilisation dans Spity
React et Tailwind CSS	Composants interactifs et interface responsive.
React Hook Form	Gestion des formulaires et des erreurs associées aux champs.
Zod	Validation des entrées, sorties et formulaires.
Drizzle ORM	Requêtes typées et migrations versionnées.
MariaDB 11.4	Contraintes relationnelles et transactions.
Jest et Testing Library	Règles pures et comportement visible des composants.
Playwright	Parcours navigateur sur l'artefact réellement livré.
Docker Compose	Environnements locaux, de test et de production reproductibles.

Le détail des couches, des routes et des décisions est conservé dans mon [document d'architecture](#).

4. Les données et les règles métier

Le modèle de données contient deux types de profils, les équipements, les lieux, les demandes de partenariat, les événements et les inscriptions. J'ai utilisé des clés étrangères avec suppression en cascade lorsque les données dépendent directement d'un utilisateur ou d'un événement. J'ai aussi ajouté des contraintes uniques pour empêcher les doublons fonctionnels.

Une demande de partenariat possède une `pairKey` stable. Elle représente la paire d'utilisateurs indépendamment de l'ordre expéditeur/destinataire. La contrainte unique en base empêche deux demandes concurrentes pour la même paire. L'API interdit aussi l'auto-demande et vérifie que le destinataire a activé la recherche de partenaire.

Pour les événements, le point le plus sensible est la capacité. Une simple vérification dans l'interface ne suffit pas : deux grimpeurs peuvent cliquer en même temps sur la dernière place. J'ai donc placé la règle dans une transaction MariaDB. La ligne de l'événement est verrouillée, le nombre d'inscriptions actives est recalculé, puis l'inscription est créée ou réactivée.

Le test d'intégration envoie volontairement deux inscriptions concurrentes sur la dernière place. J'attends une seule réussite et un refus pour l'autre demande. Ce scénario vérifie le comportement de MariaDB et de Drizzle ensemble, ce qu'un test unitaire ne pourrait pas garantir.

Pour chaque mutation, j'applique le même ordre de contrôle :

1. vérifier l'origine de la requête ;
2. récupérer et valider la session ;
3. vérifier le rôle ;
4. valider les paramètres et le corps avec Zod ;
5. vérifier l'autorisation sur la ressource ;
6. appliquer la règle métier ;
7. écrire avec Drizzle ;
8. valider la réponse avant de l'envoyer.

Cet ordre m'évite de charger ou de modifier une ressource avant d'avoir contrôlé l'identité. Il rend aussi les Route Handlers plus faciles à relire. Pour les listes, j'ai limité les résultats du matching à 100 profils dans le prototype. Je sais qu'une pagination et des index supplémentaires seront nécessaires à plus grande échelle, mais je préfère garder cette limite explicite plutôt que d'introduire une requête non bornée.

5. Ma démarche de sécurité

Je n'ai pas traité la sécurité comme une vérification ajoutée à la fin. J'ai utilisé l'OWASP Top 10:2025 comme grille de lecture et j'ai relié chaque catégorie à une mesure concrète du projet. La [matrice OWASP](#) contient les fichiers concernés, les tests et les risques résiduels.

Les mots de passe sont hachés avec bcrypt. La session est stockée dans un cookie `HttpOnly`, `SameSite=Lax` et `Secure` en production. Les endpoints sensibles vérifient l'origine de la requête. Les entrées sont validées par Zod et je n'utilise pas de SQL brut dans le code applicatif. Les rôles `grimpeur` et `club` sont contrôlés côté serveur, même lorsque l'interface masque déjà une commande.

J'ai ajouté un quota sur la connexion. Le scénario d'intégration envoie onze échecs avec la même identité réseau. Les dix premières réponses sont des `401`, puis la onzième devient une `429` avec `Retry-After` et `X-RateLimit-Remaining: 0`. Ce test me permet de montrer que la protection existe au niveau HTTP et pas seulement dans une fonction isolée.

J'ai aussi configuré les en-têtes de sécurité : CSP, HSTS en production, `X-Frame-Options`, `X-Content-Type-Options`, politique de référent et permissions. La CSP contient encore `unsafe-inline` pour rester compatible avec le rendu actuel de Next.js. Je ne présente pas ce point comme idéal : avant une mise en production publique, je devrai étudier une stratégie avec nonce ou SRI.

Deux alertes modérées concernent le PostCSS embarqué par Next.js. La correction automatique proposée par npm impose un downgrade majeur incohérent. J'ai documenté le risque au lieu de forcer une mise à jour dangereuse. La CI reste bloquante dès le niveau haut ou critique.

Les autres limites que j'assume pour le prototype sont le quota en mémoire, l'absence de révocation serveur d'une session signée et l'absence de centralisation des alertes. Sur une architecture multi-instance, je remplacerais le quota local par Redis et j'ajouterais un identifiant de session révocable.

6. L'interface et l'accessibilité

J'ai voulu éviter une application qui fonctionne seulement dans le cas nominal. Les écrans asynchrones possèdent des états de chargement, des états vides et des messages de succès ou d'erreur. Les formulaires associent leurs erreurs aux champs. Les messages qui changent après une action utilisent `aria-live` ou `role="alert"`.

J'ai retenu le RGAA 4.1.2 parce que le projet est présenté dans un contexte français. Je l'ai complété avec axe, Lighthouse et des vérifications manuelles. Les audits authentifiés couvrent dix états de l'application. Les trois pages publiques atteignent 100 en accessibilité dans Lighthouse.

Un exemple concret de correction concerne le composant `EmptyState`. Son titre visuel était rendu avec un paragraphe. Visuellement, le résultat semblait correct, mais un lecteur d'écran ne pouvait pas l'utiliser comme titre de section. J'ai remplacé ce paragraphe par un `h2` et ajouté un test sémantique. Cette anomalie est enregistrée sous `BUG-A11Y-005`.

J'ai également vérifié :

- le parcours complet au clavier ;
- la présence d'un focus visible ;
- l'ordre logique des titres ;
- les libellés des boutons qui contiennent seulement une icône ;
- la préférence `prefers-reduced-motion` ;
- le reflow à 360 pixels ;
- l'absence de défilement horizontal incohérent ;
- les contrastes des textes et contrôles principaux.

Les annexes montrent le dashboard, le matching, la gestion des événements et le profil à 390 pixels. Pour moi, ces captures ne remplacent pas les tests : elles servent à rendre le résultat visible au jury. Les rapports axe, Lighthouse et Playwright restent la preuve reproductible.

7. Comment j'ai construit la stratégie de tests

J'ai séparé les tests en trois niveaux. Les tests unitaires sont rapides et isolent les règles. Les tests d'intégration démarrent MariaDB et traversent les API. La recette Playwright pilote les parcours depuis le navigateur sur un build standalone.

Au début, ma couverture Jest portait sur huit modules sélectionnés et affichait environ 96 %. Ce chiffre était correct pour ce périmètre, mais il ne représentait pas toute l'application. Lorsque j'ai exécuté une mesure contradictoire sur tout `src/`, je n'avais que 20,23 % des lignes. J'ai donc corrigé la configuration au lieu de conserver un pourcentage avantageux mais incomplet.

J'ai ajouté des tests pour les pages principales, les deux variantes du dashboard, le répertoire de lieux, les formulaires de profil, l'inventaire, le matching, les partenariats, les événements et le contrat Drizzle. Le résultat actuel est :

Indicateur global	Résultat	Seuil bloquant
Lignes et instructions	62,56 %	60 %
Branches	77,67 %	75 %
Fonctions	55,06 %	55 %
Tests	126/126	100 % réussis

Les seuils sont enregistrés dans `jest.config.ts`. Une baisse sous l'un de ces niveaux fait échouer `npm run test:coverage` et le job `Quality gates`. Le rapport complet est conservé comme artefact GitHub Actions.

La recette navigateur contient six scénarios qui couvrent F01 à F10. Je n'utilise ni retry ni test ignoré. Cette décision rend les échecs plus visibles, mais elle évite de masquer une instabilité. Les Server Components asynchrones et les cookies de production sont surtout vérifiés à ce niveau, car ils sont plus représentatifs dans l'application assemblée.

Je sais que 62,56 % ne signifie pas que tout est testé. Les repositories Drizzle, plusieurs Route Handlers et certaines fiches de lieux restent moins couverts que les composants et les règles pures. Ils constituent la prochaine priorité de renforcement.

8. Mon pipeline d'intégration et de livraison

Chaque push sur `develop` déclenche quatre jobs en parallèle : qualité, intégration MariaDB/accessibilité, Lighthouse et recette BC02. Le staging ne démarre que lorsque ces quatre portes sont vertes. J'ai choisi cette organisation pour que chaque job donne un diagnostic lisible et pour éviter qu'un test lent empêche de voir rapidement une erreur de typage.

Le job qualité exécute l'installation avec `npm ci`, ESLint, TypeScript, les 126 tests avec couverture, l'audit npm, la validation des trois fichiers Compose et le build Next.js. Le job d'intégration crée une base MariaDB vide, applique toutes les migrations puis exécute les scénarios HTTP. La recette construit le standalone utilisé dans l'image Docker avant de lancer Playwright.

J'ai rencontré un échec important sur le run `29749715001`. La recette distante utilisait `next dev` sur un cache froid et compilait des routes pendant les scénarios. Le staging a été bloqué, ce qui était le comportement attendu, mais le test ne portait pas exactement sur l'artefact livré. J'ai remplacé ce serveur par le build standalone et aligné l'URL de boucle locale sur `localhost` afin de conserver le cookie `Secure`. Le run suivant a réussi sans retry.

Le versionnement suit des commits conventionnels et une branche `develop`. La release `v0.1.0` contient un tag, un changelog, un bundle, un manifeste et des sommes SHA-256. Les images staging sont identifiées par le SHA du commit. Je peux donc relier une capture, un rapport et un conteneur à une révision précise.

La capture de l'annexe A6 montre le pipeline complet : cinq jobs réussis et six artefacts. Ce résultat est plus important qu'un simple build local, car GitHub repart d'un runner neuf avec une installation verrouillée.

9. Les anomalies qui m'ont fait progresser

J'ai conservé les anomalies réelles dans le [plan de correction](#). Je ne voulais pas inventer des bogues pour remplir le dossier. Certaines erreurs venaient du produit, d'autres du harnais, mais elles ont toutes été reproduites et retestées.

Le parseur de matériel m'a fourni un premier exemple. La saisie `Beal Joker corde 60 m 9,1 mm turquoise` était coupée au niveau de la virgule décimale. Le parseur traitait toutes les virgules comme des séparateurs de liste. J'ai corrigé l'expression de découpage et ajouté un test qui conserve le diamètre et la couleur.

Dans le matching, un grimpeur qui avait désactivé la recherche restait joignable par une nouvelle demande. La cause était un accès public commun à l'annuaire et à l'historique. J'ai séparé l'accès d'éligibilité de l'accès nécessaire pour afficher une ancienne relation.

Plusieurs erreurs Playwright étaient des erreurs de ciblage : un libellé correspondait à la fois au champ mot de passe et au bouton d'affichage, ou un rôle `alert` correspondait aussi à l'annonceur de route Next.js. Je les ai corrigées en ciblant la relation exacte entre le champ et son erreur, sans modifier artificiellement l'interface.

Enfin, GitHub signalait la dépréciation du runtime Node.js 20 pour certaines actions. J'ai migré `setup-node` et `upload-artifact` vers leur version 6. Le run suivant ne contenait plus l'annotation. Cette correction n'a pas changé le produit, mais elle améliore la maintenabilité de la chaîne.

La règle que j'ai retenue est simple : je ne ferme une anomalie qu'après le test ciblé et le pipeline complet. Je ne baisse pas un seuil, je n'ajoute pas un retry par défaut et je ne désactive pas une protection pour obtenir un résultat vert.

10. Mon bilan et la suite du projet

Ce bloc m'a obligé à aller plus loin que le développement des écrans. J'ai dû rendre l'environnement reproductible, définir des seuils, tester la concurrence, documenter les risques, analyser des échecs et produire une version que je peux relier à un SHA.

Le point dont je suis le plus satisfait est la cohérence du parcours grimpeur/club. Une inscription à un événement n'est pas seulement un bouton : elle dépend d'une session, d'un rôle, d'une date, d'une capacité, d'une transaction et d'une restitution différente selon l'utilisateur. C'est ce type de chaîne que je veux montrer pendant la soutenance.

Je garde aussi un regard critique sur le résultat. Spity n'est pas encore prêt pour une ouverture publique. Il manque notamment :

- une session pilote formalisée avec des utilisateurs externes ;
- l'export et la suppression autonome des données personnelles ;
- la récupération de compte ;
- une stratégie de sauvegarde automatisée et testée régulièrement ;
- un quota distribué et des sessions révocables ;
- la persistance complète du fil social, de la carte et des topos ;
- davantage de tests sur la couche serveur.

Ma prochaine étape prioritaire est une session utilisateur complémentaire. L'instance publique, les comptes de démonstration et le manuel rendent déjà le logiciel manipulable en autonomie ; je veux maintenant faire exécuter le parcours à deux personnes sans les guider, mesurer les blocages et ouvrir de nouvelles anomalies si nécessaire.

Ensuite, je renforcerai les parcours RGPD et les tests des Route Handlers avant d'ajouter de nouvelles fonctions. Pour la partie produit, la carte et les topos sont les évolutions les plus cohérentes avec le sujet, mais elles devront être accompagnées d'une stratégie de modération et de fiabilité des informations.

Pour conclure, ce dossier représente le travail que j'ai réellement réalisé et les décisions que je peux expliquer. Les résultats chiffrés, les commits, les captures et les runs CI sont présents pour permettre au jury de vérifier mes affirmations. Je préfère conserver une limite explicite plutôt que de présenter comme terminé un élément que je n'ai pas encore validé.

Index des critères et des preuves BC02

1. Objet

Cet index reprend chaque critère de la grille d'évaluation du bloc 2 du titre Expert en développement logiciel RNCP39583. Il indique où le jury peut trouver l'explication, la réalisation technique et le résultat vérifié.

Sources officielles utilisées :

- Référentiel Expert en développement logiciel RNCP39583, bloc 2, pages 7 à 9 ;
- grille d'évaluation BC02 datée du 10 octobre 2024.

Statuts : **couvert** signifie que l'explication, la réalisation et une vérification suffisante sont présentes ; **partiel** signale une preuve existante mais incomplète ; **à compléter** signifie que le seuil littéral de la grille n'est pas encore atteint.

2. Résultat global

Verdict : 9 compétences sur 9, 16 livrables sur 16 et 26 critères sur 26 sont couverts par une preuve vérifiable.

Compétence	Livrable	Statut	Preuve principale
C2.1.1	Protocole de déploiement continu et critères qualité/performance	Couvert	Environnements et déploiement
C2.1.2	Protocole d'intégration continue	Couvert	Protocole CI
C2.2.1	Architecture maintenable et prototype fonctionnel	Couvert	Architecture du prototype
C2.2.2	Jeu de tests unitaires	Couvert	Harnais unitaire et audit officiel
C2.2.3	Sécurité et accessibilité	Couvert	OWASP et RGAA
C2.2.4	Historique et dernière version fiable	Couvert	Versions et déploiements
C2.3.1	Cahier de recettes	Couvert	Cahier F01 à F10
C2.3.2	Plan de correction des bogues	Couvert	Registre des anomalies
C2.4.1	Trois manuels d'exploitation	Couvert	Déploiement , utilisation , maintenance

3. C2.1.1 - Environnements, qualité et déploiement

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Le protocole de déploiement continu est explicité.	Séquence de validation, staging immuable, migration, santé, promotion et retour arrière décrite dans les sections 8 et 9.	Document C2.1.1 , workflow CI , workflow release	Couvert
L'environnement de développement est détaillé.	Node.js 22, npm 10, Next.js, TypeScript, MariaDB, Docker Compose, variables et commandes de démarrage sont précisés.	Document C2.1.1 , package.json , Compose local	Couvert
Les outils permettent d'identifier compilateur, serveur d'application et gestion de sources.	TypeScript est vérifié par <code>tsc</code> , Next.js/Node fournit le serveur, Git et GitHub assurent les sources ; Drizzle, Jest, Playwright, Lighthouse et Docker complètent la chaîne.	Table des outils , README	Couvert

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Le protocole définit les différentes séquences de déploiement.	Conditions d'entrée, sauvegarde, migration one-shot, démarrage, contrôle, promotion et rollback sont ordonnés.	Manuel de déploiement , runbook de release	Couvert
Les critères qualité/performance répondent aux exigences du projet.	Seuils lint, typage, couverture globale, vulnérabilités, accessibilité et Lighthouse sont bloquants.	Seuils C2.1.1 , run CI <code>29827790355</code> , rapports locaux <code>.lighthouseci</code> et <code>coverage</code>	Couvert

4. C2.1.2 - Intégration continue

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Le protocole d'intégration continue est explicité clairement.	Déclencheurs, environnement, permissions, stratégie de branches, revue et traitement d'échec sont documentés.	Protocole CI	Couvert
Le protocole définit les séquences d'intégration.	Les jobs qualité, MariaDB/accessibilité, Lighthouse et recette standalone précèdent obligatoirement le staging.	Workflow CI , run réussi 29827790355	Couvert

5. C2.2.1 - Architecture et prototype

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les bonnes pratiques de développement sont respectées.	Architecture par feature, TypeScript strict, composants serveur, validation Zod, dépôts Drizzle et composants UI partagés.	Architecture , src/features , src/components/ui	Couvert
Le prototype est fonctionnel et répond aux besoins identifiés.	Les rôles grimpeur et club réalisent inscription, profil, matching, partenariats et événements.	Périmètre et user stories , recette	Couvert
Le prototype met en œuvre un ensemble cohérent de fonctionnalités et les user stories.	F01 à F10 sont reliées à dix user stories et six scénarios navigateur.	Traçabilité fonctionnelle , architecture section 5	Couvert
Les composants de l'interface sont présents et fonctionnels.	Formulaires, filtres, onglets, boutons, navigation responsive, états vides et retours d'action sont pilotés par Playwright.	Captures visuelles , tests/acceptance	Couvert
Le prototype satisfait les exigences de sécurité.	Sessions HttpOnly, contrôle d'origine, autorisations par rôle, Zod, bcrypt, rate limiting et en-têtes de sécurité sont actifs et testés.	Matrice OWASP , scénario <code>REC-F09-001</code>	Couvert

6. C2.2.2 - Tests unitaires

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les tests unitaires couvrent la majorité du code développé.	Les 23 suites et 126 tests mesurent tous les fichiers <code>src/</code> et couvrent 62,56 % des lignes/instructions, 77,67 % des branches et 55,06 % des fonctions. Les seuils globaux sont bloquants dans Jest et la CI.	Harnais et périmètre , run CI 29827790355	Couvert

Limite déclarée : la majorité du code est couverte, mais les dépôts Drizzle, les Route Handlers et plusieurs Server Components restent moins couverts que les règles métier et composants interactifs.

7. C2.2.3 - Sécurité et accessibilité

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les mesures couvrent les dix principales failles OWASP.	La matrice OWASP Top 10:2025 associe chaque risque à une mesure, un fichier, un test et un risque résiduel.	Matrice OWASP , <code>npm run security:audit</code> , scénario REC-F09-001	Couvert
Le référentiel d'accessibilité est présenté et justifié.	Le RGAA 4.1.2 est retenu pour le contexte français et complété par des tests axe, Lighthouse et manuels.	Audit RGAA	Couvert
Le prototype répond aux exigences du référentiel établi.	Dix états authentifiés et trois pages publiques obtiennent 100 % en accessibilité automatisée ; clavier, lien d'évitement, mouvement réduit et reflow 360 px sont vérifiés.	Résultats RGAA , <code>.accessibility/summary.json</code> , captures mobiles	Couvert

8. C2.2.4 - Versions et version fiable

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Un système de gestion de versions est utilisé.	Git, branches <code>develop</code> / <code>main</code> , Conventional Commits, SemVer, tags et images par SHA sont utilisés.	Gestion des versions , CHANGELOG.md , historique Git	Couvert
Les évolutions du prototype sont tracées.	Le changelog, les commits, les migrations et le registre d'anomalies relient chaque évolution à une preuve.	CHANGELOG.md , drizzle , bogues	Couvert
Le logiciel est fonctionnel et manipulable en autonomie.	L'instance HTTPS publique, deux comptes de démonstration prêts à l'emploi et le manuel pas à pas permettent de réaliser les parcours grimpeur et club sans installation ni accès au code. La connexion, la session, le rendu desktop/mobile et la santé de la révision déployée ont été vérifiés.	Application jury , santé , manuel utilisateur	Couvert

Une session pilote formalisée avec des utilisateurs externes reste recommandée pour mesurer la facilité d'usage et ouvrir d'éventuelles améliorations. Elle complète la preuve de manipulation autonome, mais n'est pas présentée comme une session déjà réalisée.

9. C2.3.1 - Cahier de recettes

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Le cahier reprend l'ensemble des fonctionnalités attendues.	Les fonctions F01 à F10 sont toutes reliées à un scénario, des préconditions, des actions et des résultats.	Cahier de recettes , bc02-recipe.spec.ts	Couvert
Les tests fonctionnels, structurels et de sécurité sont conformes au plan.	Six scénarios Playwright vérifient rôles, données, capacité, sécurité, clavier et mobile sur MariaDB migrée.	Run 29819189642, artefact <code>acceptance-c1cb0f1...</code> , résultats	Couvert

10. C2.3.2 - Correction des bogues

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les bogues sont détectés, qualifiés et traités.	Sept anomalies réelles comportent impact, sévérité, priorité, cause, correction, SHA et retest.	Registre des anomalies	Couvert
Une analyse des améliorations est réalisée pour chaque test en échec.	Les erreurs produit et les défauts du harnais sont distingués ; chaque échec Playwright observé possède une prévention.	Analyse des échecs	Couvert
Les corrections garantissent le bon fonctionnement attendu.	Aucun seuil ni exigence n'a été abaissé ; la CI suivante et le staging doivent être verts pour fermer une anomalie bloquante.	Runs <code>29749715001</code> bloqué puis <code>29750556481</code> réussi, commits <code>2941a44</code> et <code>689e59d</code>	Couvert

11. C2.4.1 - Documentation d'exploitation

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les manuels sont rédigés avec clarté.	Trois documents séparés s'adressent respectivement à l'exploitant, l'utilisateur et le mainteneur ; commandes, rôles, diagnostics et checklists sont explicités.	Déploiement , utilisation , mise à jour	Couvert
La documentation décrit les choix de technologies et de langages.	TypeScript, Next.js, React, Zod, Drizzle, MariaDB, Node.js, Docker, GitHub Actions et les outils de tests sont justifiés par couche.	Choix technologiques , repères de maintenance	Couvert

12. Preuves visuelles

Les neuf [annexes visuelles](#) rassemblent quatre captures du prototype, les rapports Jest et Playwright, puis trois preuves GitHub : historique des commits, pipeline CI complet et release stable. Ce regroupement évite de répéter les mêmes images dans le corps du dossier.

13. Preuves distantes pérennes

Preuve	URL	Résultat
CI initiale complète de recette	Run 29747713909	Quatre portes et staging réussis.
Blocage réel du staging	Run 29749715001	Recette en échec, staging ignoré.
Retest standalone	Run 29750556481	Cinq jobs réussis et images publiées.
Release stable	Spity v0.1.0	Bundle, manifeste, somme SHA-256 et images versionnées.
Audit final du dossier	Run 29819189642	Cinq jobs réussis sur le commit du PDF synthétique.
Couverture globale et dossier actualisé	Run 29827790355	126 tests, seuils globaux, cinq jobs et staging réussis.
Instance jury et audit de production	Run 29988586736	Cinq jobs réussis, dépendances de production sans vulnérabilité et images staging publiées.

14. Points restant à organiser

Ces améliorations ne remettent pas en cause la couverture des vingt-six critères, mais doivent être présentées honnêtement au jury :

- conduire une session pilote formalisée avec au moins deux utilisateurs externes pour mesurer l'ergonomie réelle ;
- poursuivre progressivement la couverture des couches serveur au-delà du seuil global de 60 % ;
- conserver des captures des protections de branches GitHub si elles sont activées dans l'interface distante ;
- exécuter périodiquement un exercice de restauration sur un environnement isolé ;
- poursuivre les fonctions hors prototype : carte interactive, fil social persistant, topos et parcours RGPD autonomes.

Annexes visuelles et preuves d'exécution - BC02

Objet des annexes

Ces annexes regroupent les preuves visuelles essentielles sans alourdir le corps du dossier. Les captures produit et les rapports locaux sont générés avec `npm run docs:capture`. Les captures GitHub publiques sont régénérables avec `npm run docs:capture:github`.

Annexe	Preuve	Critères principalement concernés
A1	Tableau de bord grimpeur	C2.2.1, C2.3.1
A2	Recherche et filtres de matching	C2.2.1, C2.3.1
A3	Gestion des événements par un club	C2.2.1, C2.3.1
A4	Profil mobile	C2.2.1, C2.2.3
A5	Historique Git de <code>develop</code>	C2.2.4, C2.3.2
A6	Pipeline CI et staging réussis	C2.1.2, C2.2.2, C2.2.3, C2.3.1
A7	Release stable <code>v0.1.0</code>	C2.2.4, C2.4.1
A8	Rapport de couverture globale Jest	C2.2.2
A9	Rapport de recette Playwright	C2.3.1

A1 - Tableau de bord grimpeur

La vue d'accueil authentifiée centralise le profil, les recommandations de partenaires, le matériel et les événements. Elle prouve l'intégration cohérente des fonctions F02, F03, F06 et F07.

The screenshot shows the Spity user dashboard for a climber named Lina M. The interface is dark-themed with green accents. At the top, a navigation bar includes the Spity logo, a 'Grimpeur' tab, a 'Feed' button, and links for 'Partenaires', 'Demandes', 'Lieux', 'Événements', 'Profil', and 'Déconnexion'. The main content area features a large header with 'Bonjour Lina M.' and a description of the community. Below this are three summary cards: '2 DISCIPLINES', '1 OBJETS DÉCLARÉS', and '1 PROFILS DISPONIBLES'. The central section displays a profile card for Lina M. with options for 'Session', 'Partenaire', and 'Lieu'. To the right, a 'Profil actif' card shows her email, disciplines ('bloc', 'voie'), and equipment ('1 x Grand Rebound'). At the bottom, a 'Session bloc ce soir vers 19h' announcement is visible, and a 'Prochains rendez-vous publiés par les clubs' section lists an event at Curis.

MVP feed

Bonjour Lina M.

Un premier fil communautaire pour connecter les grimpeurs, les clubs, les lieux et les sessions à venir.

Matching local Topos vivants Événements clubs

2 DISCIPLINES

1 OBJETS DÉCLARÉS

1 PROFILS DISPONIBLES

Lina M.
Partagez une session ou trouvez un partenaire.

Session Partenaire Lieu

Profil actif
lina.demo@spity.local

Disciplines
bloc voie

Matériel
1 x Grand Rebound

Événements proches
Prochains rendez-vous publiés par les clubs.

Sortie falaise découverte à Curis
Club Alpin Lyon

Lina M.
Arkose Lyon · Bloc · 6b
Il y a 18 min

Session bloc ce soir vers 19h. Je cherche quelqu'un pour travailler les profils déversants et filmer

A2 - Recherche de partenaires

L'annuaire permet de combiner recherche textuelle, discipline, niveau, disponibilité et environnement, puis d'envoyer une demande de partenariat.

Spity

Grimpeur

Feed

Partenaires

Demandes

Lieux

Événements

Profil

Profil

Déconnexion

Matching grimpeurs

Trouver un partenaire

Filtrez les profils disponibles puis envoyez une demande de partenariat suivie dans Spity.

Suivre mes demandes

1

PROFILS DISPONIBLES

1

RÉSULTATS

Nom ou localisation

Discipline

Niveau

Disponibilité

Environnement

Q

Lyon, Camille...

Toutes les pratiques

Tous les niveaux

Toutes les disponibilités

Tous les environnements

Réinitialiser les filtres

N

Nassim B.

6a+

Villeurbanne

Voie et grande voie, disponible sur les pauses midi et le week-end.

voie

trad

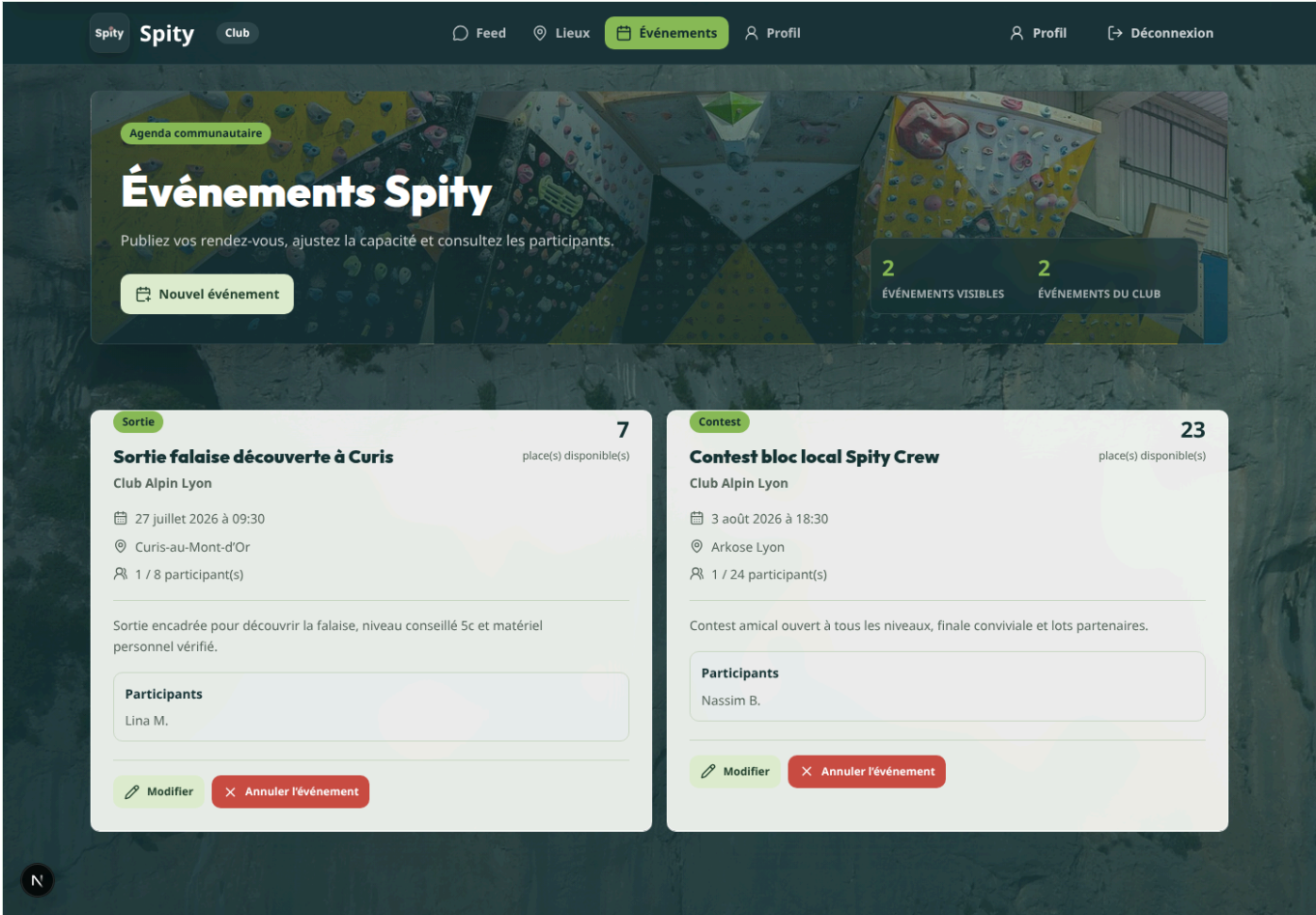
mixed

Demande en attente

N

A3 - Gestion des événements

Le rôle club peut publier, modifier et annuler un événement, suivre sa capacité et consulter les participants inscrits.



A4 - Reflow mobile du profil

La vue profil reste utilisable à 390 pixels de large. Elle complète les audits automatisés par une preuve du reflow et de l'absence de défilement horizontal incohérent.



A5 - Historique Git de la branche develop

La [branche develop](#) expose des commits courts et conventionnels. Les évolutions de couverture, de documentation et de correction restent reliées à un SHA.

PlatformSolutionsResourcesOpen SourceEnterprisePricing

Search or jump to.../Sign inSign up

Dorianyloj / spityPublic

NotificationsFork0Star0

<> CodeIssuesPull requests1ActionsProjectsSecurity and qualityInsights

Commits

develop

All usersAll time

Commits on Jul 21, 2026

docs(bc02): record successful coverage pipeline

Dorianyloj committed

dd49932

<>

docs(bc02): prove global unit coverage

Dorianyloj committed

b9e34c2

<>

test(coverage): enforce global unit coverage

Dorianyloj committed

9bb4efa

<>

docs(bc02): audit official evaluation coverage

Dorianyloj committed

e54bf20

<>

docs(bc02): shorten final certification dossier

Dorianyloj committed

c1cb0f1

<>

Commits on Jul 20, 2026

docs(bc02): assemble final certification dossier

Dorianyloj committed

0ea776d

<>

docs(bc02): record standalone acceptance evidence [skip ci]

Dorianyloj committed

d683d08

<>

test(ci): run acceptance on standalone build

Dorianyloj committed

689e59d

<>

docs(bc02): add exploitation manuals

Dorianyloj committed

c24d505

<>

A6 - Pipeline CI complet et staging

Le [run GitHub Actions 29828300315](#) réussit les portes qualité, MariaDB/accessibilité, Lighthouse et recette BC02 avant le staging. Six artefacts sont publiés, dont le rapport de couverture globale.

Platform

Solutions

Resources

Open Source

Enterprise

Pricing

Search or jump to...

Sign in

Sign up

Dorianyloj / spity

Public

Notifications

Fork 0

Star 0

<> Code

Issues

Pull requests 1

Actions

Projects

Security and quality

Insights

Continuous integration

docs(bc02): record successful coverage pipeline #16

Sign in to view logs

Summary

All jobs

Run details

Usage

Workflow file

Triggered via push 30 minutes ago

Dorianyloj pushed [dd49932](#) [develop](#)

Status

Success

Total duration

6m 8s

Artifacts

6

ci.yml

on: push

Quality gates1m 39s

MariaDB integration tests3m 40s

Lighthouse thresholds1m 31s

BC02 acceptance recipe2m 1s

Deploy verified staging i...2m 21s

Artifacts

Produced during runtime

Name	Size	Digest
acceptance- dd499329d62d099858681a831d4a367cd26b shf0	206 KB	sha256: ee06383a3e392b615a1d33717f9cf2aee06... Copy

A7 - Release stable Spity v0.1.0

La [release v0.1.0](#) relie un tag, un SHA, un changelog et quatre artefacts téléchargeables. Elle constitue la version stable du prototype présenté.

PlatformSolutionsResourcesOpen SourceEnterprisePricing

Search or jump to.../Sign inSign up

Dorianyloj / spityPublic

NotificationsFork0Star0

<> CodeIssuesPull requests1ActionsProjectsSecurity and qualityInsights

Releases / v0.1.0

Spity v0.1.0LatestCompare

github-actions released this yesterdayv0.1.00bdd4e7

What's Changed

- feat(ui): add landing page and design system by @Dorianyloj in #1
- chore(project): clean up foundation setup by @Dorianyloj in #2
- feat(auth): add email password session foundation by @Dorianyloj in #3
- feat(profile): add onboarding and profile APIs by @Dorianyloj in #4
- feat(auth): add user-facing login and register pages by @Dorianyloj in #5
- Feat/app shell by @Dorianyloj in #6
- Feat/profile equipment inventory by @Dorianyloj in #8

New Contributors

- @Dorianyloj made their first contribution in #1

Full Changelog: <https://github.com/Dorianyloj/spity/commits/v0.1.0>

Contributors



Dorianyloj

Assets4

A8 - Couverture globale du code avec Jest

Le rapport HTML mesure tout `src/`, y compris les fichiers sans test. Il confirme une couverture majoritaire avec 62,56 % des lignes et instructions, 77,67 % des branches et 55,06 % des fonctions.

All files

62.56% Statements 6512/1040977.67% Branches 675/86955.06% Functions 125/22762.56% Lines 6512/10409

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File ^		Statements ^		Branches ^		Functions ^		Lines ^	
app		85.08%	445/523	78.57%	11/14	50%	2/4	85.08%	
app/api/auth/login		0%	0/98	0%	0/1	0%	0/1	0%	

A9 - Recette Playwright F01 à F10

Le rapport Playwright présente les six scénarios de recette exécutés, sans échec, retry ni test ignoré. Ces scénarios couvrent les fonctions F01 à F10 ainsi que les contrôles fonctionnels, structurels, de sécurité et d'accessibilité.

Q Search tests	All 6	✓ Passed 6	Failed 0	Flaky 0	Skipped 0	🔄	⚙️
Project: chromium-desktop		20/07/2026 16:24:19 Total time: 14.5s					
▼ bc02-recipe.spec.ts							
✓ Cahier de recettes BC02 - fonctions F01 à F10 › REC-F01-001 - inscription, onboarding, déconnexion et reconnexion		2.7s					
chromium-desktop							
bc02-recipe.spec.ts:167							
✓ Cahier de recettes BC02 - fonctions F01 à F10 › REC-F02-001 - consultation grimpeur et création du profil club		1.7s					
chromium-desktop							
bc02-recipe.spec.ts:194							
✓ Cahier de recettes BC02 - fonctions F01 à F10 › REC-F03-F04-001 - filtres de matching, acceptation et refus		3.4s					
chromium-desktop							
bc02-recipe.spec.ts:218							
✓ Cahier de recettes BC02 - fonctions F01 à F10 › REC-F05-F08-001 - cycle complet d'un événement et suivi des participants		3.2s					
chromium-desktop							
bc02-recipe.spec.ts:266							
✓ Cahier de recettes BC02 - fonctions F01 à F10 › REC-F09-001 - authentification, rôle, origine et validation bloquent les accès invalides		714ms					
chromium-desktop							
bc02-recipe.spec.ts:322							
✓ Cahier de recettes BC02 - fonctions F01 à F10 › REC-F10-001 - structure, navigation clavier et affichage mobile		745ms					
chromium-desktop							
bc02-recipe.spec.ts:343							

Traçabilité des fichiers

Les manifestes [annexes/captures/manifest.json](#) et [annexes/github/manifest.json](#) enregistrent la date, la source et le fichier de chaque capture. Le manifeste du livrable conserve séparément les empreintes SHA-256 du HTML, du PDF et des documents assemblés.